

TaskFlow

Plan de trabajo - Reparto estilo Jira

Grupo 5 - Manejo de estado con GetX

IF0009 - Desarrollo de Software IV

Universidad de Costa Rica - Sede del Sur

Laboratorio: Manejo de Estado en Flutter

Fecha de elaboracion: 2026-09-08

En una frase

Los 6 integrantes construyen la MISMA app (TaskFlow) usando GetX. El trabajo se divide en 1 epica y 6 historias de usuario independientes, una por persona, cada una en su propia rama de Git. Este documento define quien hace que, en que rama, con que criterios de aceptacion y en que orden se integra.

Numeros del plan

EPICA	TF-0	HISTORIAS	6	STORY POINTS	26	RAMAS	6 feature + main
-------	------	-----------	---	--------------	----	-------	------------------

Integrantes

Dev 1: _____

Dev 2: _____

Dev 3: _____

Dev 4: _____

Dev 5: _____

Dev 6: _____

Nota: los nombres 'Dev 1..6' son marcadores. Sustituyanlos por los nombres reales del grupo antes de entregar. El objetivo del laboratorio no es memorizar una biblioteca, sino comprender como fluye y se transforma el estado dentro de una aplicacion.

1. Epica

ID	TF-0
Tipo	Epica
Titulo	Desarrollar TaskFlow, un gestor de tareas con manejo de estado GetX
Objetivo	Implementar los requisitos funcionales del laboratorio usando GetX (controladores, variables observables y actualizacion reactiva) separando la logica de negocio de la interfaz.
Alcance	Registrar tarea; listar tareas; marcar pendiente/completada; eliminar tarea; tablero con total, pendientes y completadas.
Fuera de alcance	Persistencia en disco / backend, autenticacion, edicion de titulo, categorias, fechas limite.
Definicion de listo (epica)	App corre de punta a punta en Windows o Chrome; los 6 branches integrados en main; README + diagrama de flujo; pruebas del controlador en verde.

Criterios de aceptacion de la epica

- Dado que abro la app, cuando no hay tareas, entonces el tablero muestra 0 / 0 / 0 y un mensaje de lista vacia.
- Dado que escribo un titulo y presiono Agregar, entonces la tarea aparece en la lista y el total sube en 1.
- Dado que marco el checkbox de una tarea, entonces pasa a completada, se tacha el texto y el tablero se actualiza.
- Dado que elimino una tarea, entonces desaparece de la lista y los contadores se recalculan.
- Dado cualquier cambio de estado, entonces solo se reconstruyen los widgets envueltos en Obx (no toda la pantalla).

Como se reparte la epica

La epica se descompone en 6 historias verticales e independientes. Cada historia es propiedad de una sola persona y vive en su propia rama. Las dependencias entre historias se resuelven acordando primero los 'contratos' (nombres de metodos y getters del TaskController) para poder trabajar en paralelo con mocks si hace falta.

2. Resumen del reparto

ID	Dev	Historia (resumen)	Rama	SP	Depende de
TF-1	Dev 1	Base del proyecto + modelo Task + tema GetMaterialApp	feature/setup-modelo	3	-
TF-2	Dev 2	TaskController: lista observable y logica de negocio	feature/task-controller	5	TF-1
TF-3	Dev 3	Formulario para registrar una tarea	feature/formulario-registro	3	TF-2 (contrato)
TF-4	Dev 4	Lista reactiva + fila con checkbox y eliminar	feature/lista-tareas	5	TF-2 (contrato)
TF-5	Dev 5	Tablero de estadísticas (total/pendientes/completadas)	feature/tablero-stats	3	TF-2 (contrato)
TF-6	Dev 6	Integracion, Binding, pantalla, README y diagrama	feature/integracion-docs	5	TF-1..TF-5

SP = Story Points (escala Fibonacci: 1,2,3,5,8). 'Depende de (contrato)' significa que solo se necesita acordar las firmas de los metodos del TaskController, no esperar a que esten implementados: se puede avanzar contra una version simulada.

Reparto del flujo de estado (quien es dueno de cada etapa)

Etapas del flujo	Que ocurre	Responsable
1. Usuario	Escribe, toca el checkbox o el boton de borrar	Dev 3 / Dev 4
2. Evento	Se invoca addTask() / toggleTask() / deleteTask()	Dev 3 / Dev 4
3. Manejador de estado	TaskController recibe la llamada y aplica la regla de negocio	Dev 2
4. Nuevo estado	Cambia el RxList 'tasks' (.obs) -> GetX marca dependencias sucias	Dev 2
5. Actualizacion de UI	Los Obx afectados se reconstruyen con los datos nuevos	Dev 4 (lista) / Dev 5 (tablero)
Ensamblaje	Binding + GetMaterialApp + pantalla que une todo	Dev 1 / Dev 6

3. Estrategia de ramas y flujo de Git

Reglas

- main es la rama estable y protegida. Nadie hace push directo a main.
- Cada historia se desarrolla en su rama feature/* (ver tabla del punto 2).
- Se abre un Pull Request de feature/* hacia main. Requiere al menos 1 revision aprobada.
- El PR debe pasar 'flutter analyze' sin errores y 'flutter test' en verde antes de merge.
- Commits pequenos y descriptivos. Convencion: tipo(area): mensaje (ej. feat(controller): agregar toggleTask).
- Rebasar o mergear main sobre la rama antes de pedir revision, para resolver conflictos localmente.

Orden de integracion recomendado

TF-1 (setup) -> TF-2 (controller) -> TF-3 / TF-4 / TF-5 en paralelo -> TF-6 (integracion y docs)

TF-1 se mergea primero porque crea la estructura y el modelo que todos importan. TF-2 va segundo porque expone el TaskController que consumen TF-3, TF-4 y TF-5. Esas tres pueden mergearse en cualquier orden. TF-6 se mergea de ultimo: arma la pantalla final, el Binding y la documentacion.

Convencion de nombres

Ramas	feature/<descripcion-corta-en-kebab-case>
Commits	feat fix docs refactor test(<area>): <mensaje en imperativo>
PR	[TF-N] <titulo de la historia> + checklist de criterios de aceptacion
Tags	v0.1 al terminar la integracion para la demo de la exposicion

Contrato compartido del TaskController (acordar ANTES de ramificar)

```
class TaskController extends GetxController {  
  final RxList<Task> tasks = <Task>[].obs;  
  
  int get total => tasks.length;  
  int get pending => tasks.where((t) => !t.completed).length;  
  int get completed => tasks.where((t) => t.completed).length;  
  
  void addTask(String title); // ignora vacios  
  void toggleTask(String id); // alterna completed + tasks.refresh()  
  void deleteTask(String id);  
  void clearAll();  
}
```

Dev 2 es dueño de la implementacion. Dev 3/4/5 programan contra estas firmas. Si alguien necesita un cambio en el contrato, se acuerda en el grupo y se actualiza aqui.

4. Historia TF-1 - Dev 1

Yo como desarrollador quiero configurar el proyecto Flutter, la estructura de carpetas y el modelo Task, para que el resto del equipo tenga una base comun, compilable y con GetX listo para trabajar en paralelo.

ID	TF-1	SP	3	PRIORIDAD	Critica
----	------	----	---	-----------	---------

Rama: feature/setup-modelo

Archivos que toca

- pubspec.yaml (dependencia get)
- lib/main.dart
- lib/models/task.dart
- analysis_options.yaml
- estructura vacia: lib/controllers, lib/bindings, lib/screens, lib/widgets

Sub-tareas

- [] Crear el proyecto: flutter create . (plataformas windows, web).
- [] Agregar GetX: flutter pub add get.
- [] Crear la clase Task con id (unico), title (String) y completed (bool, default false).
- [] Garantizar id unico aunque se creen 2 tareas en el mismo microsegundo (timestamp + contador).
- [] Configurar main.dart con GetMaterialApp, titulo 'TaskFlow' y un ThemeData con colorScheme.fromSeed.
- [] Dejar una TaskScreen provisional con un Scaffold vacio para que la app corra.
- [] Crear los folders del punto 7 del laboratorio y subir un .gitkeep si hace falta.

Criterios de aceptacion

- Dado el repo recién clonado, cuando ejecuto 'flutter run', entonces la app abre sin errores.
- Dado pubspec.yaml, entonces 'get' aparece en dependencies.
- Dado lib/models/task.dart, entonces existe la clase Task con los 3 campos y dos instancias creadas seguidas tienen id distinto.
- Dado main.dart, entonces la raiz es GetMaterialApp (no MaterialApp).

Notas tecnicas (GetX)

Es la historia mas rapida pero la mas bloqueante: hay que terminarla y mergearla el primer bloque de tiempo. Avisar al grupo apenas este en main.

4. Historia TF-2 - Dev 2

Yo como desarrollador quiero implementar el TaskController con la lista observable de tareas y toda la logica de negocio, para que la interfaz pueda registrar, alternar y borrar tareas sin conocer como se guarda el estado.

ID	TF-2	SP	5	PRIORIDAD	Critica
----	------	----	---	-----------	---------

Rama: feature/task-controller

Archivos que toca

- lib/controllers/task_controller.dart
- test/task_controller_test.dart

Sub-tareas

- [] Crear TaskController extends GetxController.
- [] Declarar el estado central: final RxList<Task> tasks = <Task>[].obs;
- [] getters derivados: total, pending, completed (calculados sobre tasks).
- [] addTask(String title): recorta espacios, ignora si queda vacio, agrega Task nuevo.
- [] toggleTask(String id): busca por id, invierte completed y llama tasks.refresh().
- [] deleteTask(String id): tasks.removeWhere(...).
- [] clearAll(): tasks.clear().
- [] Escribir pruebas unitarias de los 4 metodos + caso de titulo vacio.
- [] Comentar en el codigo donde ocurre cada etapa del flujo (evento -> nuevo estado).

Criterios de aceptacion

- Dado un titulo valido, cuando llamo addTask, entonces total incrementa y pending incrementa.
- Dado un titulo vacio o solo espacios, cuando llamo addTask, entonces tasks no cambia.
- Dada una tarea pendiente, cuando llamo toggleTask con su id, entonces completed pasa a true y completed(getter) sube.
- Dadas 2 tareas, cuando borro una por id, entonces total baja a 1 y queda la otra.
- Dado 'flutter test', entonces todas las pruebas del controlador pasan.

Notas tecnicas (GetX)

Por que tasks.refresh() en toggleTask: al mutar una propiedad de un objeto DENTRO de la lista, la referencia de RxList no cambia, asi que GetX no detecta el cambio solo. refresh() fuerza la notificacion. Alternativa: exponer Task con campos Rx, pero para el laboratorio refresh() es mas claro.

4. Historia TF-3 - Dev 3

Yo como desarrollador quiero construir el widget de formulario para agregar una tarea nueva, para que el usuario pueda registrar una tarea escribiendo su titulo y presionando Agregar (o Enter).

ID	TF-3	SP	3	PRIORIDAD	Alta
----	------	----	---	-----------	------

Rama: feature/formulario-registro

Archivos que toca

- lib/widgets/task_form.dart

Sub-tareas

- [] Crear TaskForm (StatefulWidget) con un TextEditingController local (estado local del widget).
- [] TextField con label 'Nueva tarea' y boton FilledButton 'Agregar'.
- [] Al enviar: Get.find<TaskController>().addTask(texto), luego limpiar el campo y cerrar el teclado.
- [] Enviar tambien con la tecla Enter (onSubmitted / textInputAction).
- [] No agregar nada si el campo esta vacio (se apoya en la validacion del controller).
- [] dispose() del TextEditingController.

Criterios de aceptacion

- Dado texto en el campo, cuando toco Agregar, entonces la tarea aparece en la lista y el campo queda vacio.
- Dado el campo vacio, cuando toco Agregar, entonces no pasa nada.
- Dado texto en el campo, cuando presiono Enter, entonces se agrega igual que con el boton.

Notas tecnicas (GetX)

Punto didactico: el TextEditingController es ESTADO LOCAL (solo le importa a la caja de texto). La lista de tareas es ESTADO COMPARTIDO (vive en el controller). El widget conecta ambos.

4. Historia TF-4 - Dev 4

Yo como desarrollador quiero mostrar la lista reactiva de tareas con checkbox y accion de eliminar,
para que el usuario vea todas sus tareas y pueda marcarlas como completadas o borrarlas.

ID	TF-4	SP	5	PRIORIDAD	Alta
----	------	----	---	-----------	------

Rama: feature/lista-tareas

Archivos que toca

- lib/widgets/task_list.dart
- lib/widgets/task_tile.dart

Sub-tareas

- [] TaskList: envolver un ListView en Obx(() => ...) que lee controller.tasks.
- [] Estado vacio: si tasks.isEmpty, mostrar un mensaje centrado.
- [] TaskTile: recibe un Task; Checkbox(value: task.completed, onChanged: -> toggleTask(task.id)).
- [] Texto tachado (lineThrough) y gris cuando la tarea esta completada.
- [] Boton de eliminar (IconButton) y/o Dismissible que llama deleteTask(task.id).
- [] Usar ValueKey(task.id) en cada fila para que Flutter reordene bien.

Criterios de aceptacion

- Dado que agrego una tarea, entonces aparece al final de la lista sin recargar la pantalla.
- Dado que marco el checkbox, entonces el texto se tacha y el tablero de arriba cambia.
- Dado que toco eliminar (o deslizo la fila), entonces la tarea desaparece.
- Dado que no hay tareas, entonces se ve el mensaje de lista vacia.

Notas tecnicas (GetX)

Solo este Obx se reconstruye cuando cambia la lista; la AppBar, el formulario y el tablero no. Ese es el punto a mostrar en la exposicion: reconstruccion granular.

4. Historia TF-5 - Dev 5

Yo como desarrollador quiero crear el tablero superior con total, pendientes y completadas, para que el usuario tenga un resumen del estado de sus tareas que se actualiza en tiempo real.

ID	TF-5	SP	3	PRIORIDAD	Alta
----	------	----	---	-----------	------

Rama: feature/tablero-stats

Archivos que toca

- lib/widgets/stats_board.dart

Sub-tareas

- [] StatsBoard: un Row con 3 tarjetas, envuelto en un unico Obx.
- [] Cada tarjeta lee un getter del controller: total, pending, completed.
- [] Diseno: icono + numero grande + etiqueta; color distinto por tarjeta (neutro, naranja, verde).
- [] Usar Card con color de fondo suave (withValues(alpha: ...)).
- [] No calcular nada en el widget: los conteos vienen del controller.

Criterios de aceptacion

- Dado que hay 0 tareas, entonces el tablero muestra 0 / 0 / 0.
- Dado que agrego 2 tareas y completo 1, entonces muestra Total 2, Pendientes 1, Completadas 1.
- Dado cualquier cambio en la lista, entonces los 3 numeros se actualizan sin recargar la pantalla.

Notas tecnicas (GetX)

Los getters (total/pending/completed) se leen DENTRO del Obx: por eso GetX sabe que este widget depende de tasks y lo vuelve a pintar cuando la lista cambia.

4. Historia TF-6 - Dev 6

Yo como desarrollador quiero integrar todos los widgets en TaskScreen, configurar el Binding de dependencias y documentar el proyecto, para que la app quede armada de punta a punta y documentada segun lo que pide el laboratorio.

ID	TF-6	SP	5	PRIORIDAD	Alta
----	------	----	---	-----------	------

Rama: feature/integracion-docs

Archivos que toca

- lib/bindings/task_binding.dart
- lib/screens/task_screen.dart
- lib/main.dart (initialBinding)
- README.md
- docs/flujo-estado.(md|png)

Sub-tareas

- [] TaskBinding extends Bindings con `Get.lazyPut<TaskController>(() => TaskController())`.
- [] Conectar initialBinding: TaskBinding() en GetMaterialApp.
- [] TaskScreen: Scaffold con AppBar + Column(StatsBoard, TaskForm, Divider, Expanded(TaskList)).
- [] Boton 'limpiar todo' en la AppBar que llama controller.clearAll().
- [] Escribir README.md con TODAS las secciones del punto 9 del laboratorio.
- [] Hacer el diagrama del flujo de estado (Mermaid en el README + export a imagen).
- [] Probar el flujo completo manualmente y anotar ventajas/desventajas encontradas.
- [] Coordinar el orden de merge y resolver conflictos de integracion.
- [] Crear el tag v0.1 y preparar el guion de la demo (7 min).

Criterios de aceptacion

- Dado 'flutter run', entonces se puede agregar, marcar y borrar tareas y el tablero responde.
- Dado el README, entonces contiene: manejador usado, integrantes, como funciona, donde se guarda el estado, como se modifica, como se actualiza la UI, ventajas, desventajas, uso en proyecto grande y conclusiones.
- Dado el repositorio, entonces existe un diagrama del flujo de estado.
- Dada la rama main, entonces TF-1..TF-5 estan integradas y la app compila.

Notas tecnicas (GetX)

Dev 6 es el 'integrador'. Conviene que tambien revise los PR de los demas. Si el grupo es de menos de 6, esta historia se reparte entre Dev 1 y Dev 2.

5. Mapa del flujo de estado (para la exposicion)

El laboratorio pide identificar este flujo y senalar en el propio codigo donde ocurre cada etapa. Con GetX el mapa es:

Etapa	Que pasa en el codigo	Archivo (dueno)
Usuario	Toca el checkbox de una tarea	TaskTile (Dev 4)
Evento	onChanged: () => controller.toggleTask(task.id)	TaskTile (Dev 4)
Manejador de estado	TaskController.toggleTask() encuentra la tarea y aplica la regla	TaskController (Dev 2)
Nuevo estado	task.completed = !task.completed; tasks.refresh();	TaskController (Dev 2)
Notificacion	GetX marca sucias las dependencias del RxList 'tasks'	GetX (framework)
Actualizacion de UI	Los Obx de TaskList y StatsBoard se reconstruyen con datos nuevos	TaskList (Dev 4) / StatsBoard (Dev 5)

Diagrama (incluir en el README como Mermaid)

flowchart LR

```
U[Usuario] -->|toca checkbox / boton / escribe| W[Widget: TaskTile / TaskForm]
W -->|llama metodo| C[TaskController]
C -->|muta RxList tasks .obs| S[(Nuevo estado)]
S -->|GetX notifica| O[Obx: TaskList y StatsBoard]
O -->|rebuild granular| UI[Interfaz actualizada]
UI --> U
```

6. Tablero Jira sugerido

Columnas del tablero y en cual empieza cada historia. Mover las tarjetas conforme avanza el laboratorio.

Backlog	To Do	In Progress	Code Review	Done
(vacio)	TF-1			
	TF-2			
	TF-3			
	TF-4			
	TF-5			
	TF-6			

Cronograma del laboratorio mapeado a las historias

Bloque (del laboratorio)	Tiempo	Que hace el grupo
Investigacion rapida	20 min	Todos: responder las 7 preguntas sobre GetX. Acordar el contrato del TaskController.
Desarrollo - parte 1	25 min	Dev 1 hace TF-1 y mergea. Dev 2 arranca TF-2. Resto: esqueleto de su widget con datos falsos.
Desarrollo - parte 2	35 min	Dev 2 mergea TF-2. Dev 3/4/5 conectan sus widgets al controller real. Dev 6 arma TaskScreen + Binding.
Desarrollo - parte 3	15 min	Merge de TF-3/4/5. Dev 6 integra en TF-6, prueba el flujo completo, tag v0.1.
README y exposicion	15 min	Dev 6 lidera el README y el diagrama; el resto prepara su parte del guion (7 min).

7. Definicion de Terminado (DoD)

Una historia solo se considera 'Done' cuando cumple TODO lo siguiente:

- El codigo esta en su rama feature/* y tiene un Pull Request hacia main.
- Cumple todos sus criterios de aceptacion (marcados en el PR).
- 'flutter analyze' no reporta errores (los 'info' de lint se pueden dejar si el grupo lo acuerda).
- 'flutter test' pasa (obligatorio para TF-2; recomendable para el resto).
- Al menos otra persona del grupo reviso y aprobo el PR.
- La rama se actualizo con main y no tiene conflictos.
- Si aplica, el README o los comentarios de codigo explican la parte nueva.

8. Comandos Git para cada desarrollador

```
# --- Solo Dev 1, una vez, para crear el repositorio ---
flutter create . --org cr.ac.ucr --project-name taskflow --platforms=windows,web
flutter pub add get
git init
git add .
git commit -m "chore: estructura inicial del proyecto TaskFlow"
git branch -M main
git remote add origin <URL-DEL-REPO>
git push -u origin main
```

```
# --- Cada desarrollador, para su historia ---
git checkout main
git pull origin main
git checkout -b feature/mi-historia      # ej: feature/task-controller

# ...programar y hacer commits pequenos...
git add .
git commit -m "feat(controller): agregar addTask y toggleTask"

# antes de pedir revision: traer los cambios de main
git checkout main && git pull origin main
git checkout feature/mi-historia
git merge main      # resolver conflictos aqui si los hay

git push -u origin feature/mi-historia
# luego abrir el Pull Request en GitHub: feature/mi-historia -> main
```

Sugerencia: activar en GitHub la proteccion de la rama main (Settings > Branches) para exigir Pull Request y al menos 1 aprobacion antes de mergear.

9. Investigacion rapida - GetX (respuestas guia)

Todos los integrantes deben poder responder estas preguntas en la exposicion.

Que problema intenta resolver?

Reducir el codigo repetitivo del manejo de estado, navegacion e inyeccion de dependencias en Flutter, sin necesidad de context y con reconstruccion granular de la UI.

Como almacena o representa el estado?

En controladores (GetXController). Las variables reactivas se marcan con .obs (RxInt, RxList, etc.) y GetX guarda una lista de widgets suscritos a cada una.

Como se modifica el estado?

Llamando metodos del controlador que asignan/mutan las variables .obs (p. ej. tasks.add(...), tasks.refresh()). La UI nunca modifica el estado directamente.

Como detecta Flutter que debe reconstruir la interfaz?

El widget Obx(() => ...) registra que variables .obs se leyeron durante su build. Cuando alguna cambia, GetX marca ese Obx como sucio y Flutter lo reconstruye (solo ese).

Que ventajas ofrece?

Poca ceremonia, curva de aprendizaje corta, no depende de context, reconstruccion eficiente, incluye rutas e inyeccion de dependencias en el mismo paquete.

Que desventajas presenta?

Es un paquete 'todo en uno' con mucha 'magia' implicita; facilita malas practicas (logica en la UI, estado global); menos explicito que BLoC/Riverpod; acopla el proyecto a GetX.

La usarian en una app pequena, mediana o grande?

Muy comoda en apps pequenas y medianas o prototipos. En apps grandes con varios equipos, muchos preferimos algo mas explicito y testeable (Riverpod o BLoC), aunque GetX escala con disciplina.

Checklist antes de entregar

- [] Proyecto Flutter funcional (agregar, listar, marcar, eliminar, tablero).
- [] Codigo organizado en models / controllers / bindings / screens / widgets.
- [] README.md con las 10 secciones del laboratorio.
- [] Diagrama del flujo de estado incluido.
- [] Nombres reales de los integrantes en la portada y el README.
- [] Todos los branches integrados en main y la app compila desde cero.
- [] Cada integrante entiende toda la solucion (el docente puede preguntar a cualquiera).